Understanding the E-nose

From sensor to smell fingerprint — a technical guide for a CS intern

This document explains how an opto-electronic nose (E-nose) works — from the physical sensing principle all the way to the data you will use for AI models. It is written assuming a computer science background with no prior chemistry knowledge. Technical terms are introduced and explained when first used.

1. What is an E-nose?

An **electronic nose (E-nose)** is a device designed to mimic the human sense of smell. Just like your nose contains millions of receptor cells that each respond differently to odour molecules in the air, an E-nose contains an array of chemical sensors — each one sensitive to different types of molecules.

The key insight is this: **no single sensor identifies a molecule perfectly**, but the combined response of many sensors forms a unique pattern — like a fingerprint — for each smell. This pattern is what you will use as input to your machine learning models.

> Analogy for a CS person: think of it like an ensemble of weak classifiers. Each peptide sensor is a weak learner with partial information. Together, their outputs form a rich feature vector that can discriminate between smells.

The specific E-nose used in your project is based on **Surface Plasmon Resonance imaging (SPRi)**, a highly sensitive optical technique. It has **8 peptide sensors** (labelled S1, S105, S106, S24, S25, S34, S36, S55), each made of a different short protein chain. These sensors are all arranged on the same small gold-coated chip and can be read simultaneously.

2. How SPRi and Reflectivity Work

SPRi stands for **Surface Plasmon Resonance imaging**. The name sounds intimidating, but the principle is straightforward.

At the heart of the device is a **thin gold film**. The 8 peptide sensors are coated directly onto this gold surface. A **laser beam** is shone through a glass prism onto the back of this gold film at a very precise angle.

When the laser hits the gold film, some of its light is absorbed by the electrons in the gold (this is the 'plasmon resonance' part). The rest is reflected. A **camera** measures the intensity of the reflected light for each sensor zone — this is the **% Reflectivity** value you see on the y-axis of the sensogram.

Here is the key mechanism: **when smell molecules (VOCs) land on the peptide sensors, they slightly change the physical conditions at the gold surface**. This causes the amount of reflected light to change. The camera detects this change — and that change IS the signal. More molecules binding = bigger change in reflectivity = higher signal.

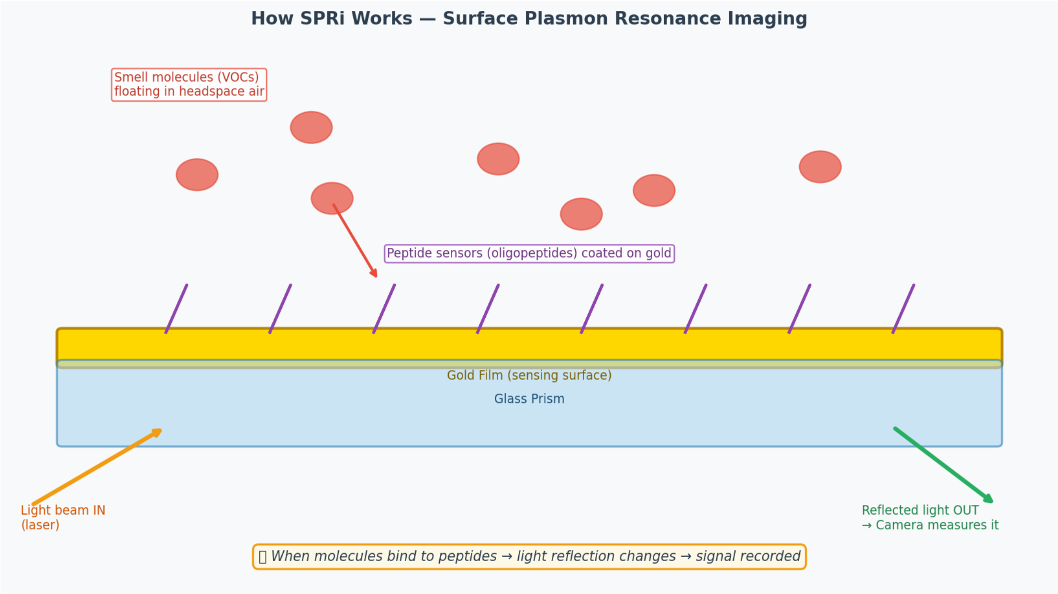

Figure 1. Simplified diagram of the SPRi principle. Light enters through the prism, reflects off the gold film where peptides are coated, and exits to a camera. When smell molecules bind to the peptides, the reflected light intensity changes.

Key point: the E-nose never 'identifies' a molecule chemically. It only measures how much each of its 8 sensors changes when exposed to the sample air. The identification is done by your AI model, which learns to associate specific patterns of sensor responses with specific smells.

3. Peptides — The Sensor Elements

Each sensor on the gold chip is a **peptide**: a short chain of amino acids (the building blocks of proteins). Different peptide sequences fold into different 3D shapes, and each shape has a different chemical affinity — meaning it is attracted to certain types of molecules more than others.

Think of each peptide as a tiny docking station with a unique shape. Some smell molecules fit well into a particular docking station (strong response), others fit poorly (weak response), and others do not fit at all (no response). The interactions involved are physical forces: electrostatic attraction, hydrogen bonds, and Van der Waals forces — not chemical reactions. This means the binding is **reversible**: molecules bind, are detected, and then leave when the air is cleaned.

Because each of the 8 peptides has a different shape and affinity, they each respond with **a different intensity to the same molecule**. This diversity is what makes the fingerprint informative. If all sensors responded identically, you could not tell molecules apart.

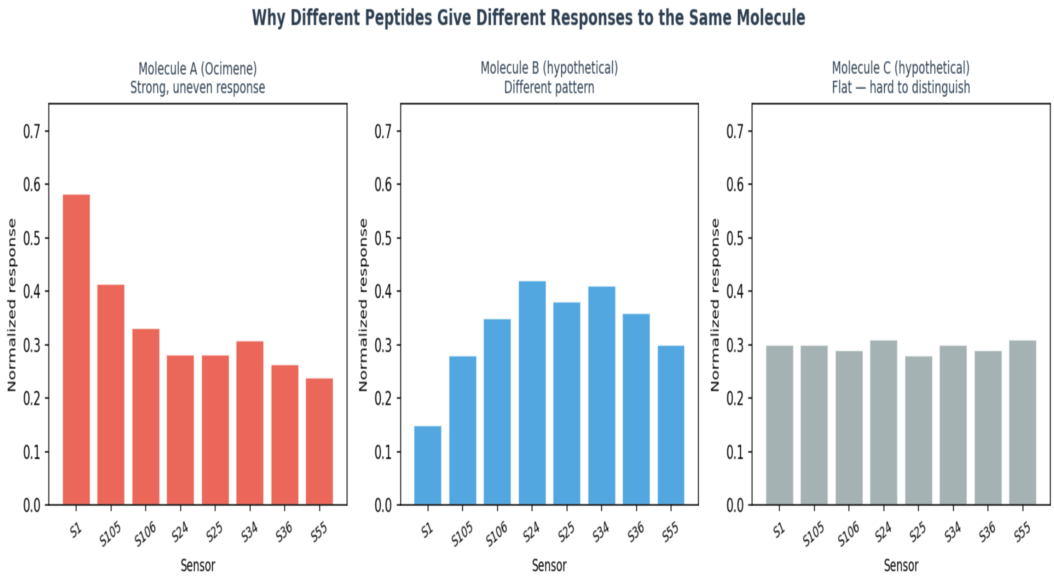

Figure 2. Illustration of why different peptide sensors give different responses. Molecule A (Ocimene, real data) produces a distinctive uneven pattern across the 8 sensors. A hypothetical Molecule B would produce a completely different pattern. A flat response (Molecule C) would be harder to discriminate.

This is directly analogous to **feature engineering** in machine learning: each sensor captures a different 'projection' of the molecule's chemical properties. Together, the 8 values form a feature vector in 8-dimensional space, where different smells cluster in different regions.

4. The Measurement Process Step by Step

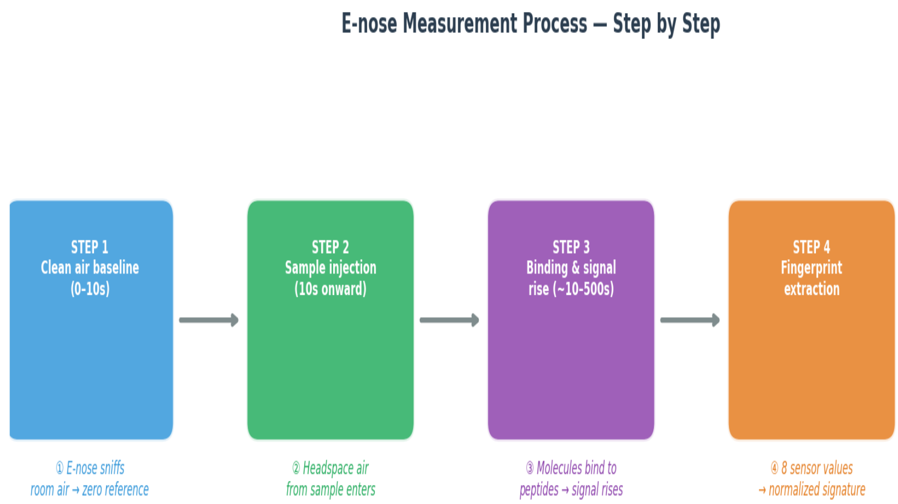

Figure 3. The four phases of a single E-nose measurement.

Step 1 — Baseline (0 to ~10 seconds)

Before any sample is introduced, the E-nose sniffs clean room air for about 10 seconds. This establishes a reference level — a zero point. All subsequent signals are measured as deviations from this baseline. This is why the sensogram starts flat at zero. In your data, this phase is labelled **section = 'baseline'**.

Step 2 — Sample Injection (~10 seconds onward)

A pump draws air from above the sample (the **headspace**) and pushes it through the E-nose at a controlled flow rate. The molecules travel through tubing and reach the gold chip where the peptide sensors are. In your data, this phase is labelled **section = 'analyte'**. You will notice a small sharp peak at the very start of this phase — this is caused by a pressure change when the valve switches from clean air to sample air. It is an artefact, not a real smell signal.

Step 3 — Binding and Signal Rise (~10 to ~1010 seconds)

As molecules flow over the sensors, they bind to the peptides. The reflectivity signal rises as more and more molecules accumulate. Different sensors rise at different rates and reach different final values, depending on their affinity for the molecule. The signal continues to rise as long as molecules are being introduced.

Step 4 — End Phase (~1010 seconds onward)

The sample flow stops. In your data this phase is labelled **section = 'na'**. At this point, the values recorded at the end of the analyte phase are used to extract the fingerprint. In a typical setup, clean air would then flush the sensors and the signal would return to baseline — ready for the next measurement.

5. Reading a Sensogram

A **sensogram** is the time-series output of the E-nose: it shows how all 8 sensor signals evolve over the full duration of a measurement. Each coloured line corresponds to one peptide sensor. The y-axis is the response value (proportional to reflectivity change), and the x-axis is time in seconds.

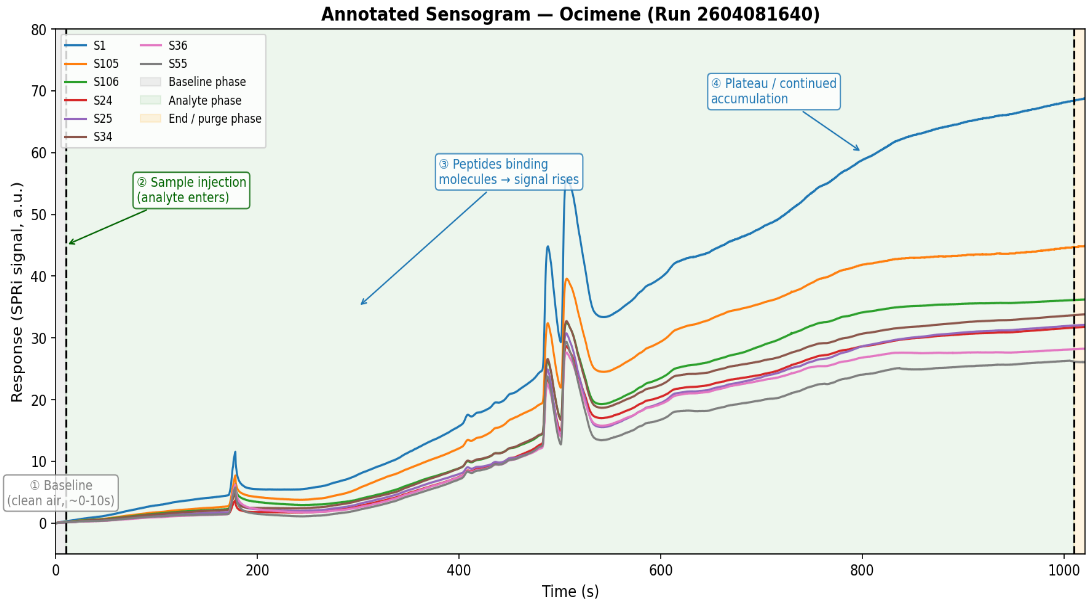

Figure 4. Annotated sensogram from a real measurement of Ocimene (Run 2604081640). The four phases are highlighted: grey = baseline, green = analyte exposure, orange = end phase. Annotations show the key events during the measurement.

What to notice in the sensogram:

The **small spike around t=180s** (visible especially in sensor S1) is the injection artefact — a pressure pulse from the valve switching. It is not caused by the molecule and should be ignored or filtered out in preprocessing.

After the injection, all sensors begin rising, but at **different rates and to different final values**. Sensor S1 (blue) rises the most steeply and reaches the highest value (~70), while S55 (grey) rises slowly and stays low (~25). This spread is what encodes the molecule's identity.

The signals do not plateau cleanly in this dataset — they continue rising toward the end of the measurement at t~1010s. This is normal for some molecules: it means binding is still occurring and equilibrium has not been reached. The **normalized signature** (fingerprint) is extracted from the final values of this curve.

The image below is from a real measurement session and shows the same type of output as your data — both the sensogram (left) and the resulting fingerprint (right).

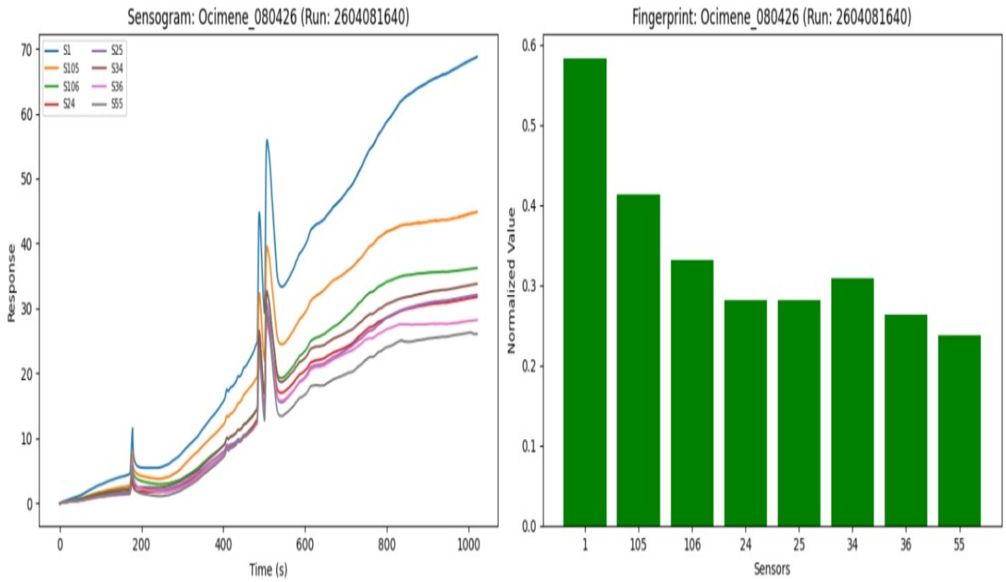

Figure 5. Example output from the E-nose software during a real session. Left: the sensogram showing 8 sensor time series. Right: the resulting smell fingerprint (normalized bar chart). Note the small spike early in the sensogram — this is the injection artefact.

6. The Smell Fingerprint (Normalized Signature)

Once the sensogram has been recorded, a single summary value is extracted for each sensor. This produces a vector of 8 numbers — the **smell fingerprint** (also called the **normalized signature** in your CSV files).

The normalization step adjusts the raw sensor values so that different measurements can be fairly compared, regardless of small variations in sample concentration, temperature, or device conditions. After normalization, each value represents the **relative contribution** of that sensor to the overall response.

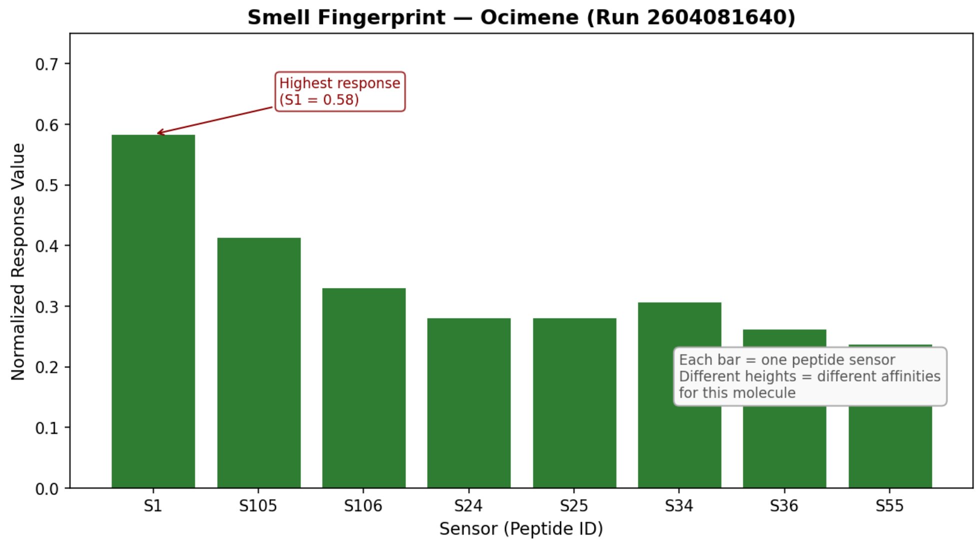

Figure 6. Smell fingerprint for Ocimene (Run 2604081640), extracted from the normalized_signatures CSV. Each bar is one peptide sensor. The pattern of bar heights is unique to this molecule — this is what your AI model will learn to recognize.

What this means for your AI work:

Each row in your **normalized_signatures.csv** file is one measurement — one smell sample. The 8 sensor columns (1, 105, 106, 24, 25, 34, 36, 55) form the **feature vector** for that sample. The run_name column gives you the label (e.g., 'Ocimene_080426'). This is your (X, y) dataset — ready for classification or clustering.

> In machine learning terms: each measurement = one data point in 8-dimensional space. Measurements of the same molecule should cluster together. Your model's job is to learn the decision boundaries between those clusters.

The **sensograms.csv** file contains the full time series — all sensor values at every timestep (~0.33s intervals) for every run. This richer data could be used for more advanced models (e.g., recurrent networks or 1D CNNs) that learn from the shape of the curve, not just its final value.

7. Temperature — Why It Matters

Your sensogram data includes a **temperature** column, recorded at every timestep. In your dataset, temperature stays very stable — around 27.7°C to 28.1°C throughout each run. This is intentional: the experiment is conducted at controlled room temperature.

Temperature matters for two reasons. First, it affects **how many molecules evaporate** from the sample into the headspace air. Warmer temperatures = more molecules in the air = stronger signal. If temperature varies between runs, the signal intensity will vary too, even for the same molecule.

Second, temperature affects the **binding affinity** of the peptide sensors themselves. At higher temperatures, molecules bind and unbind faster, which changes the shape of the sensogram curve.

For your AI work, since temperature is very stable in this dataset, it is unlikely to be a major source of variation. However, it is good practice to include it as a feature or at least check whether temperature correlates with any unexpected variation in your sensor signals — a quick correlation analysis can reveal this.

> Practical tip: if you notice that two measurements of the same molecule produce very different fingerprints, check the temperature column. A 1-2°C difference can sometimes explain unexpected signal drift.

8. From Raw Data to Your AI Input

To summarize the full pipeline from physical measurement to AI-ready data:

Stage	What happens	Your data
Sample prep	Molecule placed in vial, headspace forms above it	run_name column (molecule identity)
Baseline	E-nose sniffs clean air for ~10s	section = "baseline" rows
Injection	Headspace air pumped through E-nose	section = "analyte" rows
Binding	Molecules bind to peptides, reflectivity changes	Time-series values in sensograms.csv
Extraction	8 sensor values normalized into fingerprint	Columns 1,105,106,24,25,34,36,55 in normalized_signatures.csv
AI input	Feature vector (8 dims) + label	X = sensor columns, y = run_name

You now have all the context needed to understand where your data comes from, what each number physically represents, and what potential sources of noise or variation to watch out for. Good luck with the models!